

Sprint 04 Structured Notes: React Events and Form Inputs

0. Where You Are Before Sprint 04

Before Sprint 04, you should already know these React basics:

- React is a JavaScript library for building user interfaces.
- A React app is built from reusable components.
- JSX lets you write UI-like syntax inside JavaScript.
- Components are functions that return JSX.
- Props pass data from a parent component to a child component.
- `children` lets one component wrap other JSX.
- `map()` helps render lists.
- Stable keys help React track list items.
- Inline styles in React use JavaScript objects.
- Accessible JSX should use semantic elements, real buttons, connected labels, and meaningful alt text.

Sprint 04 starts after you can build mostly static React screens.

The missing skill is interaction.

So far, your UI can show data. In Sprint 04, your UI starts responding when the user clicks, types, and submits forms.

1. Sprint 04 Goal

The goal of Sprint 04 is to handle user actions in React.

By the end of this sprint, you should be able to:

- handle button clicks,
- handle input typing,
- handle form submission,
- stop a form from refreshing the page,
- build controlled inputs,
- build basic uncontrolled inputs,
- validate form data,
- show simple feedback,
- use `preventDefault()` correctly,
- use `stopPropagation()` when needed,

- pass extra data to event handlers,
- fix common event and form mistakes.

Simple version:

Sprint 04 teaches you how to make React interactive.

2. Big Mental Model

Before Sprint 04:

```
Data exists
↓
React displays UI
```

After Sprint 04:

```
User does something
↓
React runs an event handler
↓
State may change
↓
React updates the UI
```

Example:

```
User types in input
↓
onChange runs
↓
setName updates state
↓
React re-renders
↓
The input and screen show the new value
```

3. What Is an Event?

An event is something that happens in the browser.

Most beginner React events come from user actions.

Examples:

- clicking a button,
- typing in an input,
- submitting a form,
- selecting an option,
- checking a checkbox,
- clicking inside a card,
- changing a textarea.

Simple definition:

An event is something the user does, and React lets you respond to it.

4. React Event Names Use camelCase

In normal HTML, event names are usually lowercase.

Example in normal HTML:

```
<button onclick="doSomething()">Click</button>
```

In React JSX, event names use camelCase.

Correct React version:

```
<button onClick={handleClick}>Click</button>
```

Common React event names:

```
onClick  
onChange  
onSubmit
```

Do not write:

```
onclick  
onchange  
onsubmit
```

Beginner rule:

In React, event names use camelCase.

5. Event Handlers

An event handler is a function that runs when an event happens.

Example:

```
function handleClick() {  
  console.log("Button clicked");  
}
```

Then you connect it to JSX:

```
<button type="button" onClick={handleClick}>  
  Click me  
</button>
```

Full example:

```
export default function App() {  
  function handleClick() {  
    console.log("Button clicked");  
  }  
  
  return (  
    <main>  
      <h1>Events Practice</h1>  
  
      <button type="button" onClick={handleClick}>  
        Click me  
      </button>  
    </main>  
  );  
}
```

```
    );  
  }
```

What happens:

```
User clicks button  
↓  
React runs handleClick  
↓  
Message appears in the console
```

6. Pass the Function, Do Not Call It Immediately

This is one of the most important Sprint 04 rules.

Correct:

```
onClick={handleClick}
```

Wrong:

```
onClick={handleClick()}
```

Why?

This:

```
onClick={handleClick}
```

means:

React, run this function later when the user clicks.

This:

```
onClick={handleClick()}
```

means:

Run this function immediately while rendering the component.

That is usually wrong because the function runs before the user clicks.

Beginner rule:

Give React the function reference. Do not call the function immediately.

7. Naming Event Handlers

A common naming style is to start event handler functions with `handle`.

Examples:

```
function handleClick() {}  
function handleSubmit() {}  
function handleChange() {}  
function handleBuy() {}  
function handleDelete() {}
```

This is not required by React, but it makes code easier to read.

Good pattern:

```
function handleClick() {  
  console.log("Clicked");  
}  
  
<button onClick={handleClick}>Click</button>
```

8. Passing Extra Data to an Event Handler

Sometimes you need to pass extra data, such as an ID.

Example:

```
function ProductCard({ id, title }) {  
  function handleBuy(productId) {  
    console.log("Buying product:", productId);  
  }  
}
```

```
}

return (
  <article>
    <h2>{title}</h2>

    <button type="button" onClick={() => handleBuy(id)}>
      Buy
    </button>
  </article>
);
}
```

This part is important:

```
onClick={() => handleBuy(id)}
```

It creates a new function that React can run later.

Do not write:

```
onClick={handleBuy(id)}
```

That calls `handleBuy` immediately during render.

Beginner rule:

If you need to pass data to a handler, wrap the call in an arrow function.

9. Inline Arrow Handlers

An inline arrow handler is a small function written directly inside JSX.

Example:

```
<button onClick={() => handleBuy(id)}>
  Buy
</button>
```

This is useful when you need to pass extra data.

Another example:

```
<button onClick={() => console.log("Deleted item", item.id)}>  
  Delete  
</button>
```

Use inline arrow handlers carefully.

Good use:

```
onClick={() => handleDelete(id)}
```

Less good for large logic:

```
onClick={() => {  
  console.log("Many steps here");  
  console.log("More steps");  
  console.log("Too much logic inside JSX");  
}}
```

Beginner rule:

Small inline handlers are fine. Larger logic should go into a named function.

10. The Event Object

When an event happens, React gives your handler an event object.

The event object contains information about the event.

Example:

```
function handleChange(event) {  
  console.log(event.target.value);  
}
```

Here, `event` is the event object.

For inputs, this is very important:


```
event.target.value
```

It means:

Get the current value from the input that changed.

Full example:

```
export default function App() {  
  function handleChange(event) {  
    console.log(event.target.value);  
  }  
  
  return (  
    <main>  
      <label htmlFor="name">Name</label>  
      <input id="name" type="text" onChange={handleChange} />  
    </main>  
  );  
}
```

What happens:

```
User types A  
↓  
onChange runs  
↓  
event.target.value is "A"  
  
User types An  
↓  
onChange runs again  
↓  
event.target.value is "An"
```

11. Synthetic Events

React wraps browser events in something called a Synthetic Event.

Simple explanation:

A Synthetic Event is React's version of a browser event.

React does this so events behave more consistently across browsers.

As a beginner, you do not need to deeply memorize Synthetic Events.

You mainly need to know this:

```
function handleChange(event) {  
  console.log(event.target.value);  
}
```

The `event` object gives you useful information about what happened.

12. `onChange` for Inputs

Use `onChange` when you want to respond to input changes.

Example:

```
<input onChange={handleChange} />
```

For a text input, `onChange` runs when the user types.

Example:

```
function handleChange(event) {  
  console.log(event.target.value);  
}  
  
<input type="text" onChange={handleChange} />
```

Common uses of `onChange`:

- update state while typing,
 - show live preview,
 - validate form fields,
 - update character count,
 - enable or disable a submit button.
-

13. Controlled Inputs

A controlled input is an input where React state controls the input value.

Simple definition:

A controlled input stores its value in React state.

To build a controlled input, you need:

1. a state variable,
2. a setter function,
3. the input `value`,
4. an `onChange` handler.

Example:

```
import { useState } from "react";

export default function App() {
  const [name, setName] = useState("");

  return (
    <main>
      <h1>Controlled Input</h1>

      <label htmlFor="name">Name</label>
      <input
        id="name"
        type="text"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />

      <p>You typed: {name}</p>
    </main>
  );
}
```

Important lines:

```
const [name, setName] = useState("");
```

This creates state.

```
value={name}
```

This makes the input show the state value.

```
onChange={(event) => setName(event.target.value)}
```

This updates state when the user types.

14. Controlled Input Cycle

The controlled input cycle looks like this:

```
User types
↓
onChange runs
↓
setState updates React state
↓
React re-renders
↓
The input displays the new state value
```

Example with `name`:

```
User types "S"
↓
onChange runs
↓
setName("S")
↓
React re-renders
↓
input value becomes "S"
```

This may feel indirect at first, but it gives React full knowledge of the input value.

15. Why Controlled Inputs Are Useful

Controlled inputs are useful because React always knows the current input value.

This lets you do things like:

- show live preview,
- show character count,
- validate while typing,
- disable the submit button,
- show error messages,
- clear the form after submit,
- conditionally show UI based on the input value.

Example: live character count.

```
import { useState } from "react";

export default function MessageBox() {
  const [message, setMessage] = useState("");

  return (
    <main>
      <h1>Message Box</h1>

      <label htmlFor="message">Message</label>
      <textarea
        id="message"
        value={message}
        onChange={(event) => setMessage(event.target.value)}
      />

      <p>Characters: {message.length}</p>
    </main>
  );
}
```

Because `message` is state, React can display:

```
<p>Characters: {message.length}</p>
```

16. Form Submission in React

A form submission happens when the user submits a form.

This can happen by:

- clicking a submit button,
- pressing Enter inside some inputs.

In normal HTML, submitting a form often reloads the page.

In React SPAs, you usually do not want the page to reload.

So React forms usually use:

```
event.preventDefault();
```

17. `event.preventDefault()`

`event.preventDefault()` stops the browser's default behavior.

For forms, it stops the page refresh.

Example:

```
function handleSubmit(event) {  
  event.preventDefault();  
  console.log("Form submitted");  
}
```

Full example:

```
import { useState } from "react";  
  
export default function SignupForm() {  
  const [name, setName] = useState("");  
  
  function handleSubmit(event) {  
    event.preventDefault();  
    console.log("Submitted name:", name);  
  }  
}
```

```

    }

    return (
      <main>
        <h1>Signup</h1>

        <form onSubmit={handleSubmit}>
          <label htmlFor="name">Name</label>
          <input
            id="name"
            type="text"
            value={name}
            onChange={(event) => setName(event.target.value)}
          />

          <button type="submit">Submit</button>
        </form>
      </main>
    );
  }

```

Important part:

```
<form onSubmit={handleSubmit}>
```

This connects the submit event to `handleSubmit`.

Inside `handleSubmit`:

```
event.preventDefault();
```

This prevents the page reload.

Beginner rule:

In React forms, usually call `event.preventDefault()` inside `handleSubmit`.

18. `onSubmit` Belongs on the Form

A common beginner mistake is putting click logic only on the submit button.

Better pattern:

```
<form onSubmit={handleSubmit}>
  <button type="submit">Submit</button>
</form>
```

Why?

Because the form can be submitted in different ways.

For example:

- clicking the submit button,
- pressing Enter.

If you use `onSubmit` on the form, both can work properly.

Beginner rule:

Handle form submission with `onSubmit` on the `<form>`, not only `onClick` on the button.

19. Button Types

Buttons inside forms should usually have clear types.

Submit button:

```
<button type="submit">Submit</button>
```

Normal action button:

```
<button type="button">Click</button>
```

Why does this matter?

Inside a form, a plain `<button>` may behave like a submit button by default.

So this can accidentally submit a form:

```
<button>Cancel</button>
```


Safer version:

```
<button type="button">Cancel</button>
```

Beginner rule:

Use `type="submit"` for submit buttons and `type="button"` for normal action buttons.

20. Form Validation

Validation means checking whether user input is acceptable.

Examples:

- name is required,
- email is required,
- message cannot be empty,
- password must be long enough,
- rating must be selected.

Simple validation example:

```
const nameIsValid = name.trim().length > 0;
```

This checks whether the name has real characters after removing extra spaces.

Example:

```
import { useState } from "react";

export default function SignupForm() {
  const [name, setName] = useState("");

  const nameIsValid = name.trim().length > 0;

  function handleSubmit(event) {
    event.preventDefault();

    if (!nameIsValid) {
      console.log("Name is required");
      return;
    }
  }
}
```

```

    console.log("Submitted:", name);
  }

  return (
    <main>
      <h1>Signup</h1>

      <form onSubmit={handleSubmit}>
        <label htmlFor="name">Name</label>
        <input
          id="name"
          type="text"
          value={name}
          onChange={(event) => setName(event.target.value)}
        />

        {!nameIsValid && <p>Name is required.</p>}

        <button type="submit" disabled={!nameIsValid}>
          Submit
        </button>
      </form>
    </main>
  );
}

```

Important validation pieces:

```
const nameIsValid = name.trim().length > 0;
```

This calculates whether the input is valid.

```

if (!nameIsValid) {
  return;
}

```

This stops submission if the form is invalid.

```
<button disabled={!nameIsValid}>
```

This disables the button when the input is invalid.

21. disabled Attribute

The `disabled` attribute prevents a button or input from being used.

Example:

```
<button type="submit" disabled={!formIsValid}>  
  Submit  
</button>
```

If `formIsValid` is false, the button is disabled.

If `formIsValid` is true, the button is enabled.

Simple mental model:

```
formIsValid = true  
↓  
button works  
  
formIsValid = false  
↓  
button disabled
```

22. Multiple Controlled Inputs

A real form usually has more than one input.

Example fields:

- name,
- rating,
- message.

You can use separate state values:

```
const [name, setName] = useState("");  
const [rating, setRating] = useState("5");  
const [message, setMessage] = useState("");
```

Full example:

```
import { useState } from "react";

export default function FeedbackForm() {
  const [name, setName] = useState("");
  const [rating, setRating] = useState("5");
  const [message, setMessage] = useState("");

  const formIsValid = name.trim() !== "" && message.trim() !== "";

  function handleSubmit(event) {
    event.preventDefault();

    if (!formIsValid) {
      return;
    }

    const feedbackData = {
      name,
      rating,
      message,
    };

    console.log(feedbackData);

    setName("");
    setRating("5");
    setMessage("");
  }

  return (
    <main>
      <h1>Feedback Form</h1>

      <form onSubmit={handleSubmit}>
        <div>
          <label htmlFor="name">Name</label>
          <input
            id="name"
            type="text"
            value={name}
            onChange={(event) => setName(event.target.value)}
          />
        </div>

        <div>
```

```

    <label htmlFor="rating">Rating</label>
    <select
      id="rating"
      value={rating}
      onChange={(event) => setRating(event.target.value)}
    >
      <option value="5">5 - Excellent</option>
      <option value="4">4 - Good</option>
      <option value="3">3 - Okay</option>
      <option value="2">2 - Poor</option>
      <option value="1">1 - Bad</option>
    </select>
  </div>

  <div>
    <label htmlFor="message">Message</label>
    <textarea
      id="message"
      value={message}
      onChange={(event) => setMessage(event.target.value)}
    />
  </div>

  <p>Message length: {message.length}</p>

  <button type="submit" disabled={!formIsValid}>
    Send Feedback
  </button>
</form>
</main>
);
}

```

What this example teaches:

- text input controlled by `name`,
- select input controlled by `rating`,
- textarea controlled by `message`,
- validation with `formIsValid`,
- disabled submit button,
- form reset after submit.

23. Controlled `textarea`

A controlled textarea works like a controlled input.

Example:

```
<textarea
  value={message}
  onChange={(event) => setMessage(event.target.value)}
/>
```

Do not use normal HTML textarea content like this in React controlled forms:

```
<textarea>Hello</textarea>
```

For controlled React textareas, use `value`.

24. Controlled `select`

A controlled select also uses `value` and `onChange`.

Example:

```
<select
  value={rating}
  onChange={(event) => setRating(event.target.value)}
>
  <option value="5">5 - Excellent</option>
  <option value="4">4 - Good</option>
</select>
```

The selected option is controlled by this state:

```
const [rating, setRating] = useState("5");
```

When the user selects a different option, this runs:

```
setRating(event.target.value)
```

25. Resetting a Controlled Form

Because controlled input values live in state, you reset the form by resetting state.

Example:

```
setName("");  
setEmail("");  
setMessage("");
```

If the input value is controlled by state:

```
<input value={name} />
```

then changing `name` to an empty string clears the input.

Simple mental model:

```
state becomes empty string  
↓  
React re-renders  
↓  
input becomes empty
```

26. Uncontrolled Inputs

An uncontrolled input is an input where the DOM keeps the value, not React state.

Simple definition:

An uncontrolled input stores its current value in the DOM instead of React state.

Controlled input:

```
React state owns the value
```

Uncontrolled input:

The DOM input owns the value

27. `useRef` for Uncontrolled Inputs

To read an uncontrolled input, you can use `useRef`.

Example:

```
import { useRef } from "react";

export default function RefCodeForm() {
  const refCodeInput = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();

    const refCode = refCodeInput.current.value;

    console.log("Referral code:", refCode);
  }

  return (
    <main>
      <h1>Referral Code</h1>

      <form onSubmit={handleSubmit}>
        <label htmlFor="refCode">Referral Code</label>
        <input id="refCode" type="text" ref={refCodeInput} />

        <button type="submit">Submit</button>
      </form>
    </main>
  );
}
```

Important pieces:

```
const refCodeInput = useRef(null);
```

This creates the ref.


```
<input ref={refCodeInput} />
```

This connects the ref to the input.

```
refCodeInput.current.value
```

This reads the current input value.

28. When Are Uncontrolled Inputs Reasonable?

Uncontrolled inputs are reasonable when:

- you only need the value when the form is submitted,
- you do not need live validation,
- you do not need live preview,
- the input does not need to update the screen while typing,
- you are integrating with non-React code.

For beginner React forms, controlled inputs are usually the better default.

Beginner rule:

Use controlled inputs when the value affects the UI. Use uncontrolled inputs only when you just need to read the value later.

29. Controlled vs Uncontrolled Inputs

Topic	Controlled Input	Uncontrolled Input
Value stored in	React state	DOM input
Uses <code>value</code>	Yes	Usually no
Uses <code>onChange</code>	Yes	Not always
Uses <code>useRef</code>	Usually no	Often yes
Good for validation	Yes	Not ideal
Good for live preview	Yes	Not ideal
Good for simple submit-only value	Can work	Yes

Simple memory rule:

Controlled = React controls the value
Uncontrolled = DOM controls the value

30. `event.stopPropagation()`

Events can move upward from a child element to a parent element.

This is called bubbling.

Example:

```
<article onClick={handleCardClick}>
  <button onClick={handleButtonClick}>Buy</button>
</article>
```

If the user clicks the button, the click can also reach the article.

That means both handlers may run:

```
button click handler runs
↓
article click handler also runs
```

To stop that, use:

```
event.stopPropagation();
```

Example:

```
export default function App() {
  function handleCardClick() {
    console.log("Card clicked");
  }

  function handleButtonClick(event) {
    event.stopPropagation();
    console.log("Button clicked");
  }
}
```

```

    }

    return (
      <article onClick={handleCardClick}>
        <h2>Product</h2>

        <button type="button" onClick={handleButtonClick}>
          Buy
        </button>
      </article>
    );
  }

```

Use `stopPropagation()` only when you intentionally want to stop the event from reaching a parent.

Beginner rule:

Use `stopPropagation()` when a child click should not also trigger the parent click.

31. `preventDefault()` vs `stopPropagation()`

These two methods are different.

`preventDefault()`

Stops the browser's default behavior.

Common form example:

```

function handleSubmit(event) {
  event.preventDefault();
}

```

Meaning:

Do not refresh the page when the form submits.

`stopPropagation()`

Stops the event from bubbling to parent elements.

Common click example:

```
function handleClick(event) {  
  event.stopPropagation();  
}
```

Meaning:

Do not let this button click also trigger the parent click.

Comparison:

Method	What it stops	Common use
<code>preventDefault()</code>	Default browser action	Stop form page reload
<code>stopPropagation()</code>	Event bubbling	Stop child click from triggering parent click

32. Accessibility Habits for Forms

Sprint 04 forms should keep the accessibility habits from Sprint 03.

Important habits:

- use real form elements,
- connect labels to inputs,
- use real buttons,
- use useful button types,
- show clear validation messages,
- avoid clickable `div`s for actions.

Connected labels

Correct:

```
<label htmlFor="email">Email</label>  
<input id="email" type="email" />
```

The label uses:

```
htmlFor="email"
```

The input uses:

```
id="email"
```

They must match.

Do not write this in JSX:

```
<label for="email">Email</label>
```

Use `htmlFor`, not `for`.

Why?

Because `for` is a reserved word in JavaScript.

33. Common Sprint 04 Mistakes

Mistake 1: Lowercase event names

Wrong:

```
<form onsubmit={handleSubmit}>  
  <input onchange={handleChange} />  
</form>
```

Correct:

```
<form onSubmit={handleSubmit}>  
  <input onChange={handleChange} />  
</form>
```

Mistake 2: Calling the handler immediately

Wrong:

```
<button onClick={handleClick()}>Click</button>
```

Correct:

```
<button onClick={handleClick}>Click</button>
```

Mistake 3: Passing data without an arrow function

Wrong:

```
<button onClick={handleDelete(id)}>Delete</button>
```

Correct:

```
<button onClick={() => handleDelete(id)}>Delete</button>
```

Mistake 4: Forgetting `preventDefault()`

Wrong:

```
function handleSubmit(event) {  
  console.log(name);  
}
```

Correct:

```
function handleSubmit(event) {  
  event.preventDefault();  
  console.log(name);  
}
```

Mistake 5: Using `e` where it does not exist

Wrong:

```
<input onChange={setName(e.target.value)} />
```

Correct:

```
<input onChange={(event) => setName(event.target.value)} />
```

Mistake 6: Missing `value` on controlled input

Incomplete controlled input:

```
<input onChange={(event) => setName(event.target.value)} />
```

Better controlled input:

```
<input
  value={name}
  onChange={(event) => setName(event.target.value)}
/>
```

Mistake 7: Missing connected label

Weak accessibility:

```
<input type="email" />
```

Better:

```
<label htmlFor="email">Email</label>
<input id="email" type="email" />
```

34. Debug Task From Sprint 04

Broken code:

```
function App() {
  const [name, setName] = useState("");

  function submit(e) {
    console.log(name);
  }

  return <form onsubmit={submit()}>
    <input value={name} onChange={setName(e.target.value)} />
  </form>;
}
```

Problems

Problem 1: `useState` is not imported.

```
import { useState } from "react";
```

Problem 2: React uses `onSubmit`, not `onsubmit`.

Wrong:

```
onsubmit
```

Correct:

```
onSubmit
```

Problem 3: `submit()` is called immediately.

Wrong:

```
onSubmit={submit()}
```

Correct:

```
onSubmit={handleSubmit}
```

Problem 4: React uses `onChange`, not `onchange`.

Wrong:

```
onChange
```

Correct:

```
onChange
```

Problem 5: `e` is used where it does not exist.

Wrong:

```
onChange={setName(e.target.value)}
```

Correct:

```
onChange={(event) => setName(event.target.value)}
```

Problem 6: The form does not prevent page reload.

Add:

```
event.preventDefault();
```

Fixed version

```
import { useState } from "react";

export default function App() {
  const [name, setName] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    console.log(name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="name">Name</label>
      <input
```

```

        id="name"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />

      <button type="submit">Submit</button>
    </form>
  );
}

```

35. Full Practice Project: Contact Us Form

Goal

Build a small Contact Us form using Sprint 04 concepts.

Requirements

Use controlled inputs for:

- name,
- email,
- message.

The form should:

- show a live character count for the message,
- disable submit until all fields have text,
- use `event.preventDefault()`,
- log form data on submit,
- reset the form after submit,
- use connected labels,
- use a real submit button.

Example solution

```

import { useState } from "react";

export default function ContactForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");

```

```

const formIsValid =
  name.trim() !== "" &&
  email.trim() !== "" &&
  message.trim() !== "";

function handleSubmit(event) {
  event.preventDefault();

  if (!formIsValid) {
    return;
  }

  console.log({
    name,
    email,
    message,
  });

  setName("");
  setEmail("");
  setMessage("");
}

return (
  <main>
    <h1>Contact Us</h1>

    <form onSubmit={handleSubmit}>
      <div>
        <label htmlFor="name">Name</label>
        <input
          id="name"
          type="text"
          value={name}
          onChange={(event) => setName(event.target.value)}
        />
      </div>

      <div>
        <label htmlFor="email">Email</label>
        <input
          id="email"
          type="email"
          value={email}
          onChange={(event) => setEmail(event.target.value)}
        />
      </div>
    </form>
  </main>
)

```

```

    <div>
      <label htmlFor="message">Message</label>
      <textarea
        id="message"
        value={message}
        onChange={(event) => setMessage(event.target.value)}
      />
    </div>

    <p>Characters: {message.length}</p>

    <button type="submit" disabled={!formIsValid}>
      Send
    </button>
  </form>
</main>
);
}

```

What this project proves

This project proves that you can:

- use `useState`,
- create controlled inputs,
- use `onChange`,
- use `onSubmit`,
- use `preventDefault()`,
- validate form values,
- disable a button,
- show live UI feedback,
- reset the form,
- write accessible labels.

36. Guided Practice: Feedback Form

Build a `FeedbackForm` with:

- name,
- rating,
- message.

Rules:

- name is required,
- message is required,
- rating can default to `5`,
- submit button is disabled until name and message are not empty,
- submit logs the data,
- form resets after submit.

Starter structure:

```
import { useState } from "react";

export default function FeedbackForm() {
  const [name, setName] = useState("");
  const [rating, setRating] = useState("5");
  const [message, setMessage] = useState("");

  const formIsValid = name.trim() !== "" && message.trim() !== "";

  function handleSubmit(event) {
    event.preventDefault();

    if (!formIsValid) {
      return;
    }

    console.log({ name, rating, message });

    setName("");
    setRating("5");
    setMessage("");
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* Build the fields here */}
    </form>
  );
}
```

37. Challenge Practice: Add an Uncontrolled Referral Code

Add an uncontrolled referral code field to your form.

Use `useRef`.

Example:

```
import { useRef, useState } from "react";

export default function ContactForm() {
  const [name, setName] = useState("");
  const refCodeInput = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();

    const refCode = refCodeInput.current.value;

    console.log({
      name,
      refCode,
    });
  }

  return (
    <form onSubmit={handleSubmit}>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
        onChange={(event) => setName(event.target.value)}
      />

      <label htmlFor="refCode">Referral Code</label>
      <input id="refCode" ref={refCodeInput} />

      <button type="submit">Submit</button>
    </form>
  );
}
```

This combines:

- controlled input for `name`,
 - uncontrolled input for `refCode`,
 - form submit handling,
 - `useRef`.
-

38. Sprint 04 Revision Questions and Answers

Question 1: Why do React event names use `onClick`, not `onclick`?

Because JSX is JavaScript-like syntax, and React event names use camelCase.

Correct:

`onClick`

Wrong:

`onclick`

Question 2: Why should you write `onClick={handleClick}` instead of `onClick={handleClick()}`?

Because `onClick={handleClick}` passes the function to React so React can run it later.

`onClick={handleClick()}` calls the function immediately while the component renders.

Question 3: What does `event.preventDefault()` do in a form?

It stops the browser's default form submission behavior.

In React forms, this usually means it stops the page from reloading.

Question 4: What does `event.target.value` give you?

It gives you the current value of the input that triggered the event.

Example:

```
onChange={(event) => setName(event.target.value)}
```

Question 5: What is a controlled input?

A controlled input is an input whose value is stored and controlled by React state.

Example:

```
<input value={name} onChange={(event) => setName(event.target.value)} />
```

Question 6: Why are controlled inputs useful for validation?

Because React always knows the current input value.

That means React can check the value, show errors, disable buttons, and update the UI while the user types.

Question 7: When is an uncontrolled input reasonable?

An uncontrolled input is reasonable when you only need the value on submit and do not need live validation or live preview.

Question 8: What does `useRef` help you do with uncontrolled inputs?

`useRef` lets you access the DOM input and read its current value.

Example:


```
const inputRef = useRef(null);
const value = inputRef.current.value;
```

Question 9: Why should form inputs have connected labels?

Connected labels improve accessibility and usability.

They help users and assistive technologies understand what each input means.

Correct:

```
<label htmlFor="email">Email</label>
<input id="email" />
```

Question 10: Why should submit buttons use `type="submit"`?

Because it clearly tells the browser that the button submits the form.

This works well with the form's `onSubmit` handler.

39. Sprint 04 Self-Check List

You are ready to move to Sprint 05 when you can do these without notes:

- Explain what an event is.
- Use `onClick` on a button.
- Use `onChange` on an input.
- Use `onSubmit` on a form.
- Explain why React uses camelCase event names.
- Pass a function reference to an event handler.
- Avoid calling the handler immediately by mistake.
- Pass extra data using an inline arrow function.
- Read `event.target.value` from an input.
- Explain what a controlled input is.
- Build a controlled text input.
- Build a controlled textarea.
- Build a controlled select.
- Use `preventDefault()` inside form submission.

- Validate form values.
 - Disable a submit button when the form is invalid.
 - Reset controlled form fields after submit.
 - Explain what an uncontrolled input is.
 - Use `useRef` to read an uncontrolled input.
 - Explain when uncontrolled input is reasonable.
 - Use `stopPropagation()` when a child click should not trigger a parent click.
 - Connect labels with `htmlFor` and `id`.
 - Use real buttons with correct `type` values.
 - Fix common Sprint 04 event and form bugs.
-

40. Mini Revision Sheet

Event

Something the user does.

Examples:

```
click, type, submit, select, check
```

Event handler

A function that runs when an event happens.

```
function handleClick() {}
```

React event names

Use camelCase.

```
onClick  
onChange  
onSubmit
```

Function reference

Correct:

```
onClick={handleClick}
```

Wrong:

```
onClick={handleClick()}
```

Passing extra data

Use an arrow function.

```
onClick={() => handleDelete(id)}
```

Event object

React gives event information to the handler.

```
function handleChange(event) {  
  console.log(event.target.value);  
}
```

Controlled input

Input value is stored in React state.

```
<input value={name} onChange={(event) => setName(event.target.value)} />
```

Form submit

Use `onSubmit` on the form.

```
<form onSubmit={handleSubmit}>
```

preventDefault

Stops page reload.

```
event.preventDefault();
```

Uncontrolled input

Input value is stored in the DOM.

```
const inputRef = useRef(null);  
<input ref={inputRef} />
```

stopPropagation

Stops a child event from reaching a parent.

```
event.stopPropagation();
```

Connected label

```
<label htmlFor="email">Email</label>  
<input id="email" />
```

41. Final Beginner Explanation

Sprint 04 teaches this:

I can make React respond to users. I use `onClick` for clicks, `onChange` for input typing, and `onSubmit` for forms. I pass handler functions like `handleClick`, not `handleClick()`. I use `event.target.value` to read input values. I use controlled inputs when React state should control the form value. I use `event.preventDefault()` to stop form reloads. I can validate forms, disable submit buttons, reset fields, and use `useRef` for simple uncontrolled inputs. I can also use `stopPropagation()` when a child click should not trigger a parent click.

42. Definition of Done for Sprint 04

Sprint 04 is complete when you can:

- build a button with `onClick`,
 - build an input with `onChange`,
 - build a form with `onSubmit`,
 - explain handler reference vs handler call,
 - use an inline arrow function to pass extra data,
 - read `event.target.value`,
 - build controlled inputs,
 - explain the controlled input cycle,
 - use `preventDefault()` correctly,
 - validate form data,
 - disable a submit button,
 - reset a form after submit,
 - build a basic uncontrolled input with `useRef`,
 - explain controlled vs uncontrolled inputs,
 - use `stopPropagation()` only when needed,
 - keep forms accessible with connected labels and real buttons,
 - complete the Contact Us or Feedback Form project.
-

43. Suggested Study Order

Study Sprint 04 in this order:

1. What events are
2. camelCase event names
3. `onClick`
4. Handler reference vs handler call
5. Passing extra data with arrow functions
6. Event object
7. `event.target.value`
8. Controlled inputs
9. Controlled input cycle
10. `onSubmit`
11. `preventDefault()`
12. Form validation
13. Multiple controlled inputs
14. Controlled textarea and select
15. Resetting forms
16. Uncontrolled inputs
17. `useRef`
18. `stopPropagation()`

- 19. Accessibility habits for forms
- 20. Debug task
- 21. Contact Us or Feedback Form project

Do not try to memorize everything first.

Build the examples. Break them. Fix them.

That is how Sprint 04 becomes clear.